

Apogeo

Cómo se conectan el núcleo,
los plugins y la visualización.

Guía para el equipo de desarrollo

CONFIGURAR → SIMULAR → ANALIZAR

A/P

¿Qué construimos con Apogeo?

Una plataforma de simulación

Representa el estado de un vehículo y su evolución en el tiempo.

Modelos físicos por módulos

Permite desarrollar estructuras, propulsión y otros comportamientos por separado.

Resultados y visualización

Registra la ejecución y presenta sus resultados en datos y gráficas.

tick Una vuelta completa del ciclo.

dt El tiempo simulado que avanza en esa vuelta.

$100 \text{ ticks} \times 0,01 \text{ s} = \mathbf{1 \text{ s simulado}}$

El computador puede calcularlo en más o menos de 1 segundo real.

La arquitectura se organiza en cuatro capas



Cada parte tiene una responsabilidad

Visualización

Recibe opciones del usuario y muestra resultados y gráficas.

Núcleo

Organiza la ejecución y actualiza el estado de la simulación.

Plugins

Encapsulan los datos y el comportamiento de cada modelo físico.

Datos y contratos

Definen qué información se intercambia y en qué formato.

Las tecnologías tienen un propósito

C++17

Núcleo, plugins y ejecución de la simulación.

FlatBuffers + JSON + CSV

Estado compartido; configuración; curvas y resultados.

Python + HTTP

Servidor web y lanzamiento del proceso de simulación.

HTML + CSS + JavaScript + Chart.js

Interfaz de usuario y gráficas en el navegador.

CMake + vcpkg + GoogleTest

Compilación, dependencias y pruebas automatizadas.

Una simulación comienza con entradas claras

1 • Configurar la ejecución

Elegir módulos habilitados, opciones y cantidad de pasos.

2 • Definir el estado inicial

Posición, velocidad, masa y condiciones del entorno.

3 • Cargar los modelos

Abrir los plugins y preparar sus datos de trabajo.

4 • Empezar el ciclo

El núcleo conserva el estado y comienza a avanzar el tiempo.

| Los archivos JSON describen la ejecución; cada módulo puede cargar sus propios datos.

El núcleo coordina cada paso

SimulationEngine

Organiza el arranque, los ticks y el cierre.

PluginManager

Carga módulos y coordina su ejecución.

PhysicsIntegrator

Calcula cómo evoluciona el movimiento a partir de los aportes.

OutputManager

Registra y administra las muestras y los archivos de salida.

| El núcleo mantiene el estado común: posición, velocidad, orientación, masa y entorno.

Qué ocurre durante un tick

01 Leer el estado

02 Preparar cambios

03 Consultar los modelos

04 Reunir los aportes

05 Actualizar el movimiento

06 Avanzar y registrar

01-06

Un ciclo completo

Leer estado → preparar cambios → consultar modelos → reunir aportes → actualizar movimiento → avanzar y registrar.

Un plugin aporta una parte del modelo

Su ciclo de vida

Crear la instancia



Cargar configuración y datos



Participar en cada tick



Liberar recursos

Su relación con el núcleo

- Se compila como biblioteca dinámica.
- Se implementa en C++ y exporta una ABI de C.
- Recibe acceso al estado de simulación.
- Aporta su comportamiento sin incorporar su clase interna al núcleo.

Propulsión muestra cómo crece un módulo

Datos

Un CSV contiene curvas tabuladas de empuje e impulso específico.

Lectura

El parser comprueba el archivo y prepara los datos en memoria.

Modelo

La clase de dominio ofrece consultas e interfaces para la física.

Conexión con el núcleo

El adaptador implementa el ciclo de vida común del plugin.

| Estado actual: carga y consulta de datos listas; cálculo físico de empuje y consumo pendiente.

La web conecta al usuario con el simulador

Navegador · HTML, CSS, JavaScript y Chart.js

↓ solicitudes HTTP · ↑ estado y resultados

Servidor Python · API y gestión de ejecuciones

↓ lanza el ejecutable con una configuración

Simulador C++ · núcleo y plugins

El navegador consulta el estado y solicita resultados para construir las gráficas.

Así recorre el sistema una ejecución

El usuario prepara una ejecución

Elige condiciones iniciales y número de pasos en la interfaz.

Python prepara y lanza

Escribe la configuración y arranca el simulador C++.

El núcleo ejecuta los ticks

Coordina plugins, integración, reloj y muestreo de resultados.

La interfaz presenta los resultados

Consulta la ejecución y muestra datos en tablas y gráficas.

| La misma simulación puede iniciarse desde terminal, sin abrir la interfaz web.

Dónde trabajar según el cambio

src/core/ y src/api/

Flujo de simulación y contrato común de los plugins.

plugins/ y data/

Modelos físicos, parámetros y archivos de entrada.

tools/web_server/

Servidor, endpoints, interfaz y gráficas.

tests/ y pruebas de cada módulo

Comportamientos esperados y regresiones.

docs/specs/ y docs/

Diseño de funcionalidades, decisiones y guías de uso.

Diseñar, implementar y comprobar

01

Definir

Qué debe hacer la funcionalidad y cómo se acepta.

`functional_spec.md`

02

Diseñar

Qué capa cambia, qué datos usa y qué pasos requiere.

`technical_spec.md`
`task_list.md`

03

Validar

Implementar, compilar, probar y actualizar la documentación.

CMake · GoogleTest · Doxygen

| La spec explica el comportamiento; las pruebas lo comprueban; la API documenta cómo usarlo.

Primer recorrido para incorporarse

1 • Ubicar las piezas

Leer README y esta guía; localizar núcleo, plugins y web.

2 • Ejecutar un caso pequeño

Seguir BUILD_RUN, compilar y observar una ejecución corta.

3 • Seguir un dato

Ver cómo la configuración llega al núcleo y termina en resultados.

4 • Hacer un cambio acotado

Elegir una capa, actualizar su spec, implementar y validar.

| Al tomar una tarea: ¿qué capa cambia, qué contrato toca y cómo demuestro que funciona?